ATC'26 · Hong Kong · November 2026

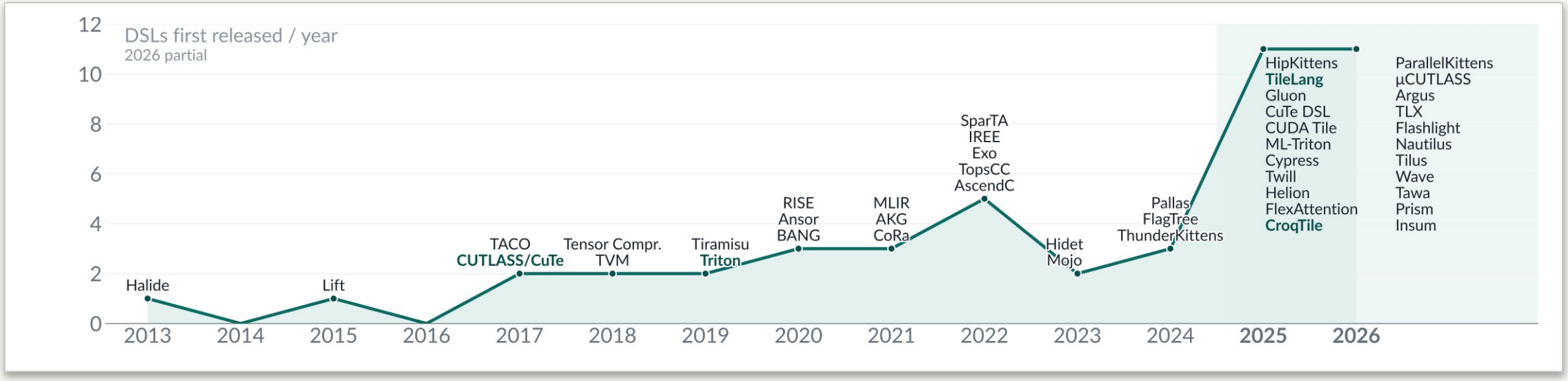
A Novel AI-Native GPU Kernel Programming Language for Autonomous Kernel Tuning

Heng Shi · Xiaofeng Guan · Sijie Wu · Enming Fan · Chenpeng Wu
Qiqi Gu · Jiahang Xie · Guoqing Bao · Jianguo Yao

Shanghai Enflame Intelligence & Shanghai Jiao Tong University

Operator work keeps growing

Kernel variants multiply.



22/46 DSLs in 2025–26



Why kernels multiply

1 **Operator variants** — MHA · MQA · GQA · MLA

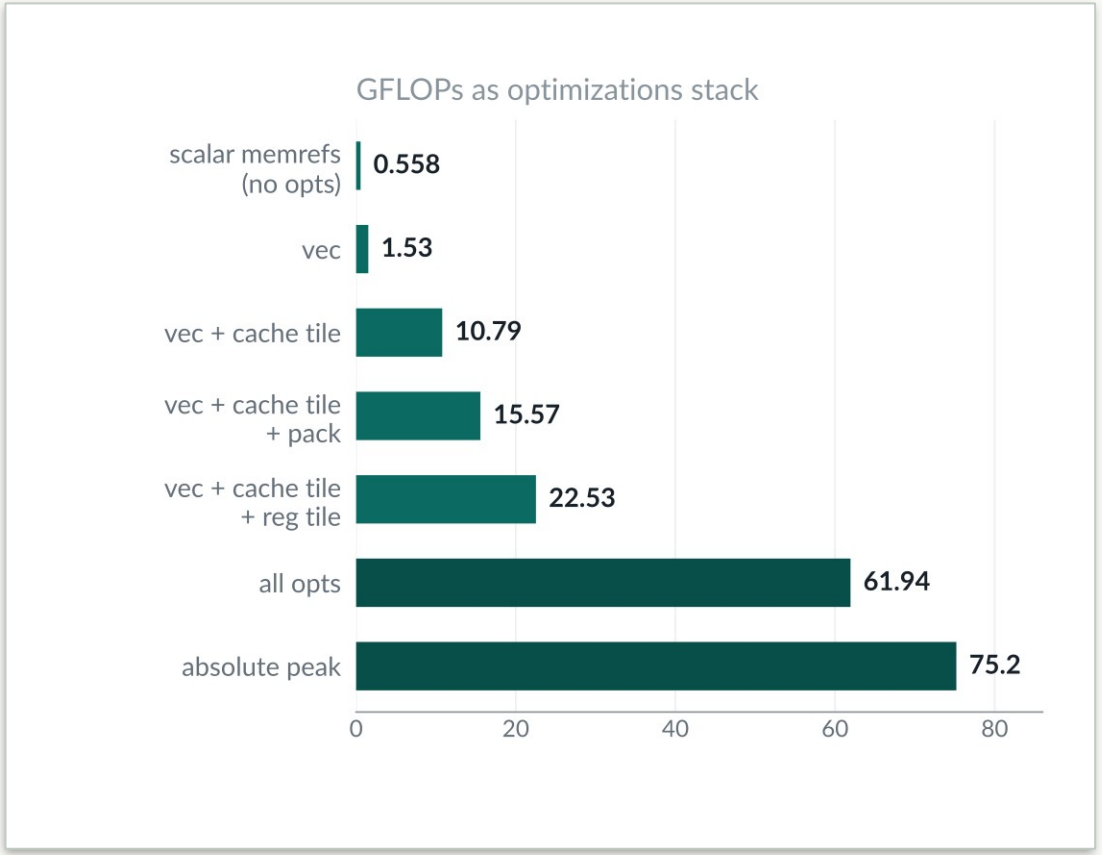
2 **Hardware diversity** — MI300X · Ascend · TPU · Trainium

3 **New GPU instructions** — Hopper wgmma → Blackwell tcgen05

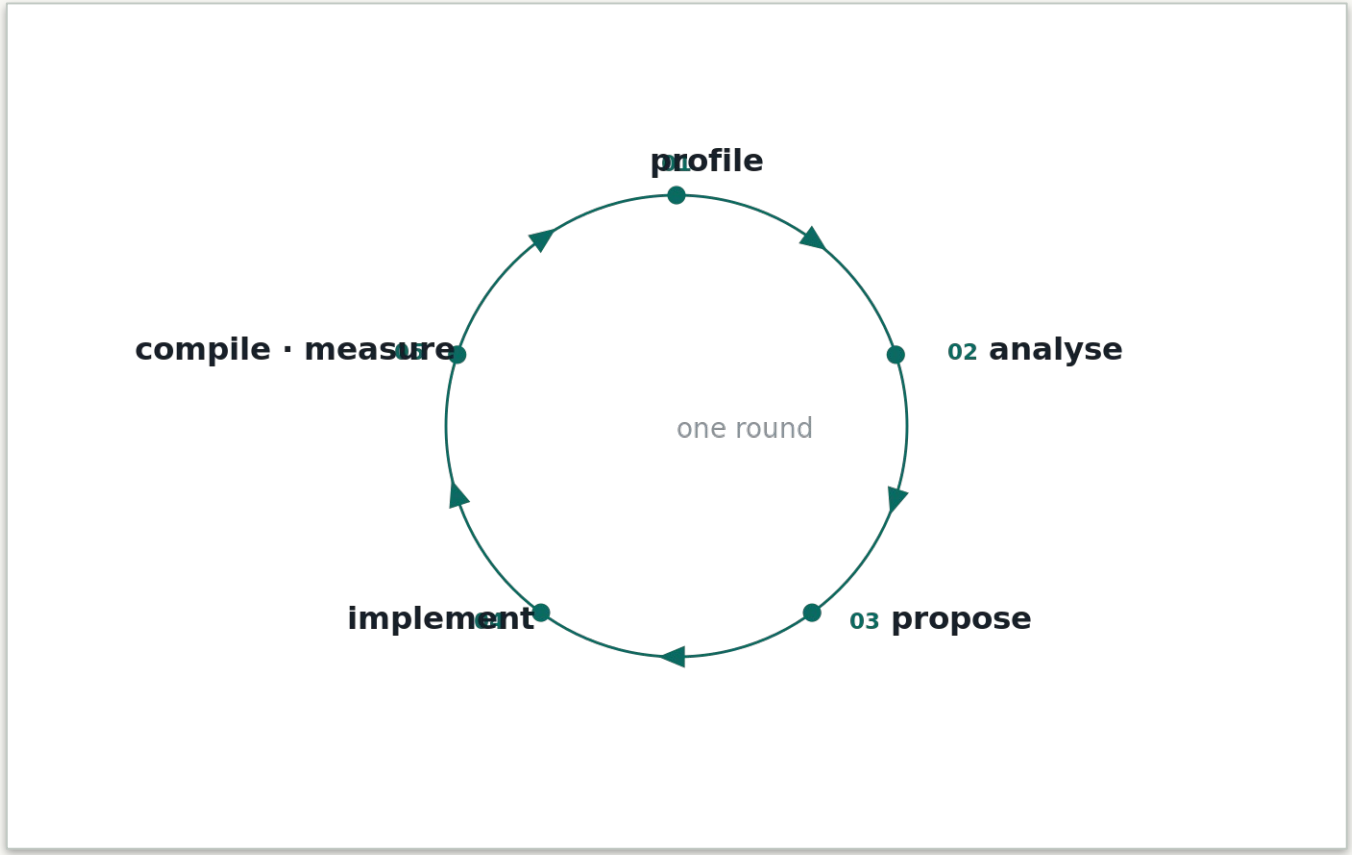
4 **Operator fusion** — Conv + BN + ReLU → one kernel

Kernel optimizations interact

Tiling	**Data reuse**
Swizzling	**Multibuffering**
Unrolling	**Vectorization**
Approximation	**Sparsity**
Fusion	**L2 bypass**
Async copies	**Warp specialization**
MMA layouts	**Clusters / DSMEM**
Scheduling	**Low-precision MMA**



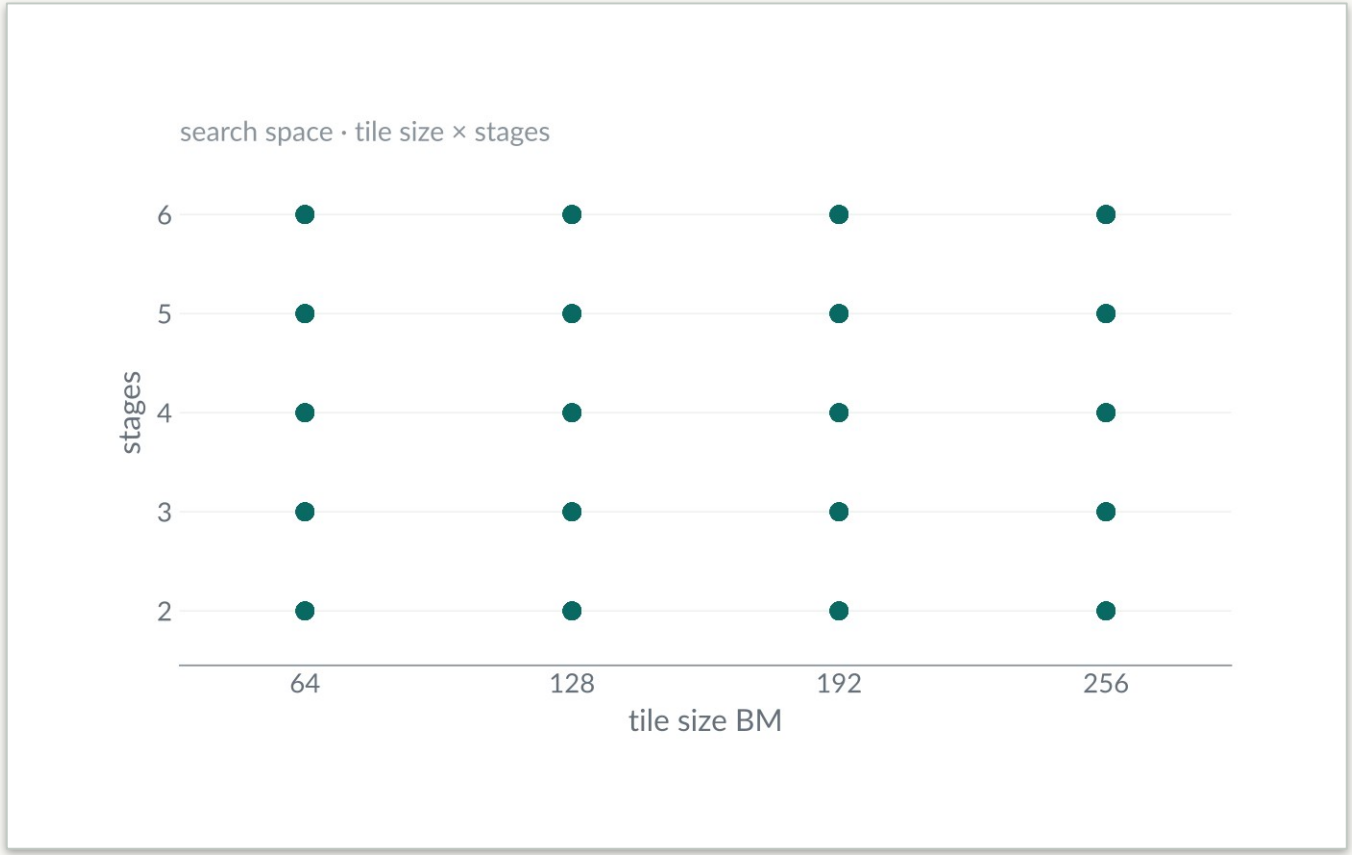
Bottleneck-driven tuning



1. Profile
2. Identify bottleneck
3. Choose structural edit
4. Implement
5. Compile · measure · keep / reject

Bottleneck → structural change

Template search



`Tile size · stages · warps · swizzle`

Fixed skeleton

Parameter search

Resource pruning + benchmarking

Search within the template

Compiler transformations

data tiling / blocking	→ **Loop tiling**
data stationary	→ **Cache staging**
multibuffering & prefetch	→ **Software pipelining**
loop unroll	→ **Unrolling**
vectorisation	→ **Vectorization**
loop fusion	→ **Fusion**
instruction scheduling	→ **Instruction scheduling**

MLIR · TVM · IREE

Schedule DSL · Halide / TVM TE

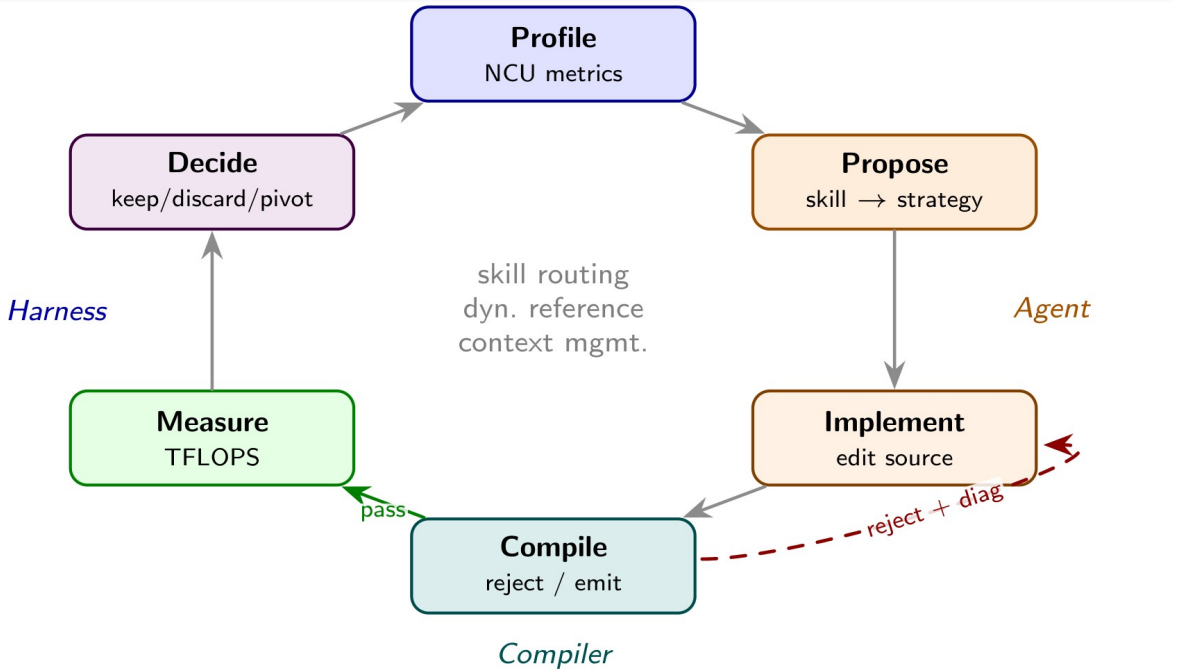
Compiler passes · loop transformations

Performance depends on exposed transformations

Agents rewrite the kernel

`Profile → edit → evaluate`

Full source edits · structural + parametric



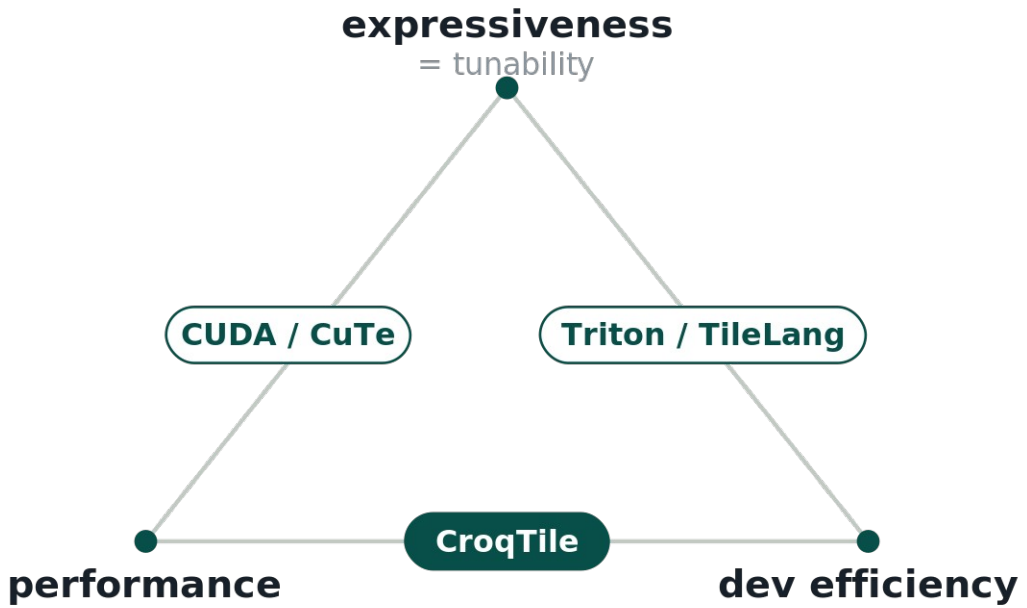
`Workflow`	Planning · search · refinement
`Evals`	Correctness · speed · generalization
`Tooling`	Profiler · compiler · benchmark
`Knowledge`	Skills · references · history

Design for agent failures

Failure	Requirement
Context pressure	Compact source
Coupled edits	Independent primitives
Opaque failures	Early, actionable feedback

Performance + cheap iteration

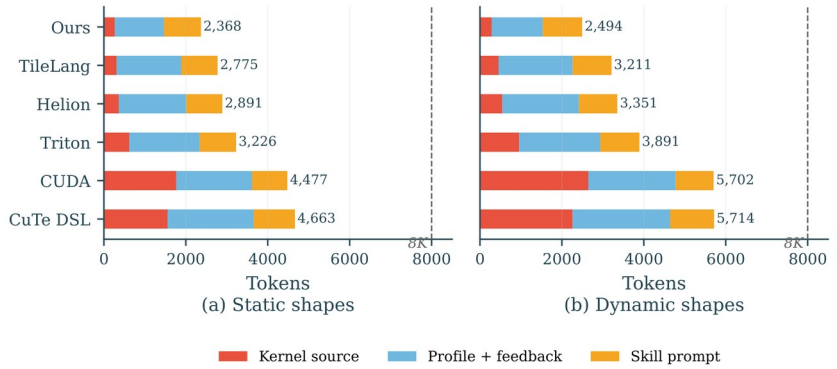
Expressiveness · performance · iteration cost

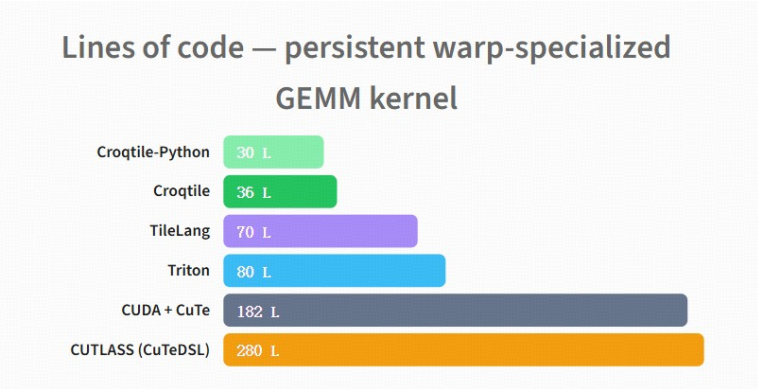


CUDA / CuTe	**Explicit control**	Long source · coupled edits
Triton / TileLang	**Compact tile operations**	Some structural controls hidden

CroqTile	**Compact + explicit**	Independent performance controls

Compact syntax preserves context





Structure

$kernel \rightarrow$ `__co__ void id (` $param^*$ `) {` $stmt^*$ `}`
$param \rightarrow$ `[`$addrsp$`]` $dtype$ `[`$shape^+$`] id`
$addrsp \rightarrow$ `global` | `shared` | `local`
$stmt \rightarrow$ $decl$ | $parallel$ | $foreach$ | $warpspec$ | dma | mma | $event$ | $sync$ | $assign$ | ...

Parallelism & Iteration

$parallel \rightarrow$ `parallel` $binds$ `:` $space$ `{` $stmt^*$ `}`
$binds \rightarrow$ `id by` $expr$ | `{ id`$^+$ `} by [`$expr^+$`]`
$space \rightarrow$ `block` | `group-`K | `cluster`
$foreach \rightarrow$ `foreach { id`$^+$ `} in [`$expr$`]` `{` $stmt^*$ `}`
$warpspec \rightarrow$ `inthreads.async (`$expr$`)` `{` $stmt^*$ `}`

Declarations

$decl \rightarrow$ `[`$addrsp$`]` $dtype$ `[`$shape^+$`] id`
$event \rightarrow$ `shared event id[`N`] (, id[`N`])`*

Data Movement

$dma \rightarrow$ `(tma|dma).copy[.async<`ev`>]` $attr^*$ ref `=>` ref
$attr \rightarrow$ `.swiz<`N`>` | `.zfill` | `.sparse`

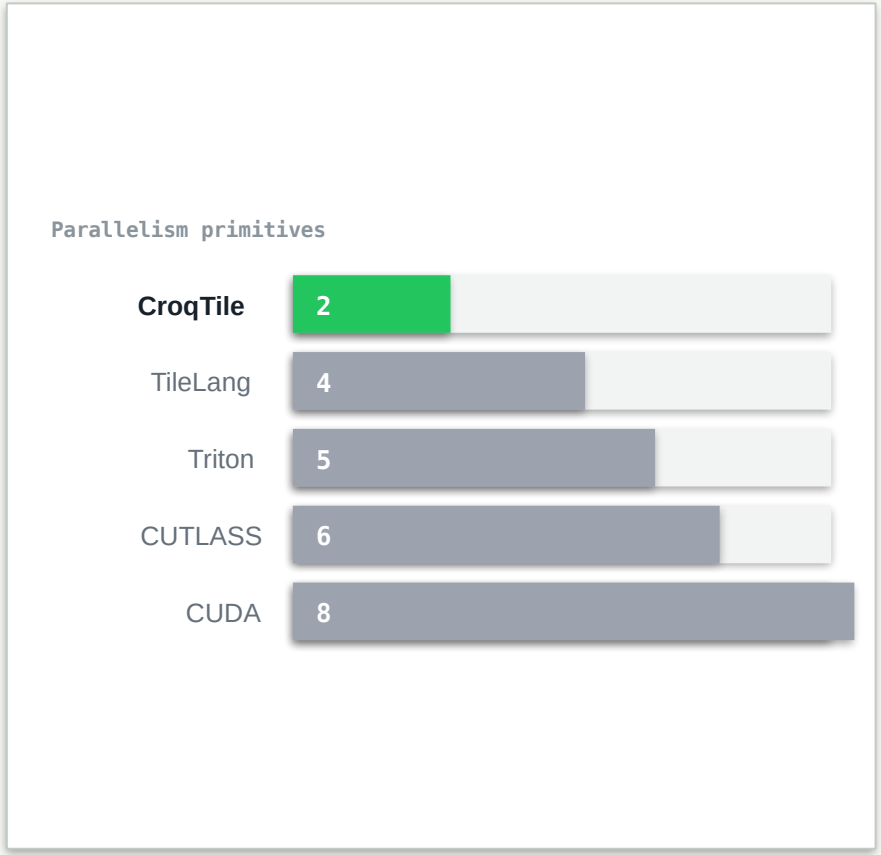
Compute

$mma \rightarrow$ `mma.`op `[.`mod`]`* ref^*
$op \rightarrow$ `fill` | `row.row` | `row.col` | `col.col` | `commit` | `store`
$mod \rightarrow$ `sp` | `scale`

Synchronization

$sync \rightarrow$ `wait id[`$expr$`]` | `trigger id[`$expr$`]`

parallel: one construct, three levels



CroqTile | TileLang | CuTe / CUTLASS | CUDA

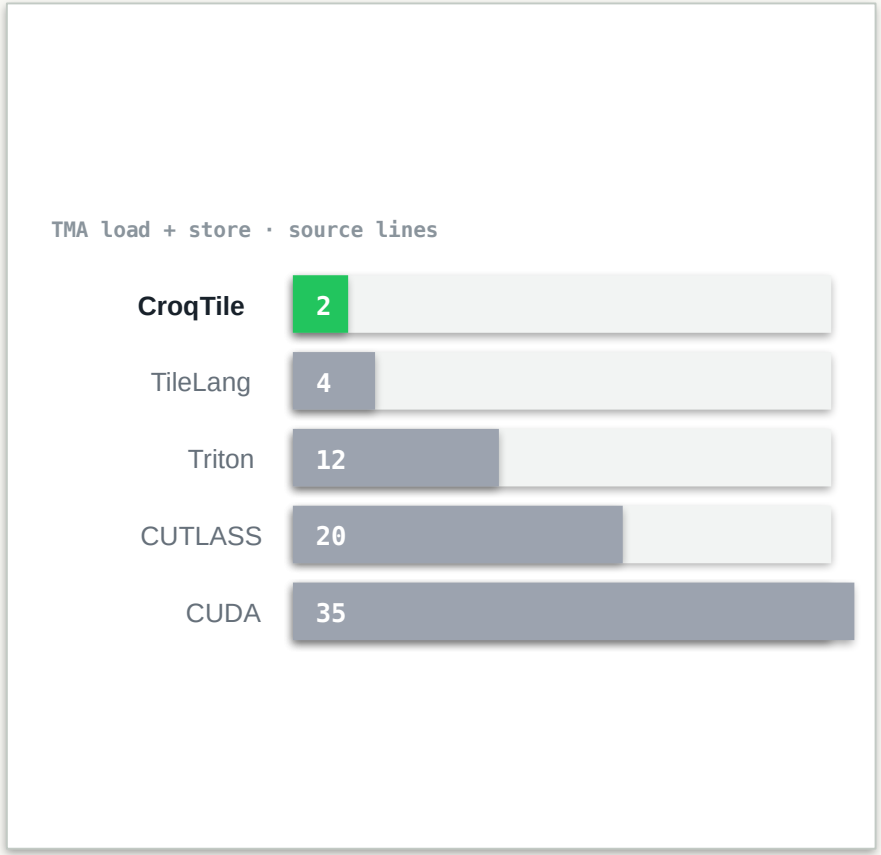
```
parallel {bm, bn}
  by [cdiv(M, WARP_M),
      cdiv(N, WARP_N)] : block {

  parallel p by 1 : group-4 {
    mma.row.row mc, ma, mb;
  }

  parallel {m, n}
    by [WARP_M, 2] : thread { ... }
}

parallel p1 by 2 : group-4 {
  inthreads.async (p1 == 0) { /*TMA*/ }
  inthreads.async (p1 == 1) { /*MMA*/ }
}
```

tma: one statement per transfer



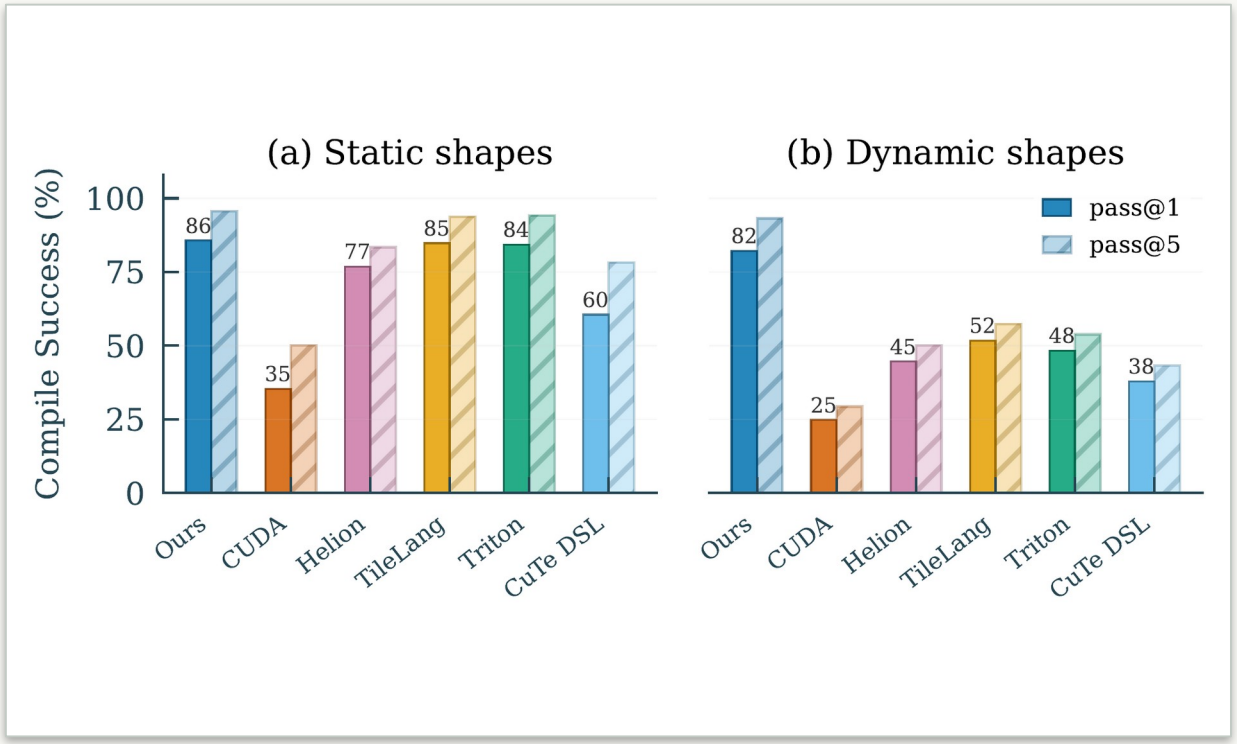
CroqTile | Triton | TileLang | CuTe / CUTLASS | CUDA

```
// TMA load: global -> shared (1 line)
tma.copy.swiz<128>
  lhs.subspan(WARP_M, TILE_K)
    .at(bm, iv_k) => lhs_s;

// TMA store: shared -> global (1 line)
tma.copy out_s =>
  output.subspan(WARP_M, WARP_N)
    .at(bm, bn);
```

Compile success across shapes

pass@1 / pass@5 · static + dynamic



Fewer edit sites → smaller repairs

1 **Static shapes**

85.7% pass@1

2 **Dynamic shapes**

82.1% pass@1 · Triton 48%

3 **Type system**

Symbolic shapes · compile-time inference

Diagnostics that guide repair

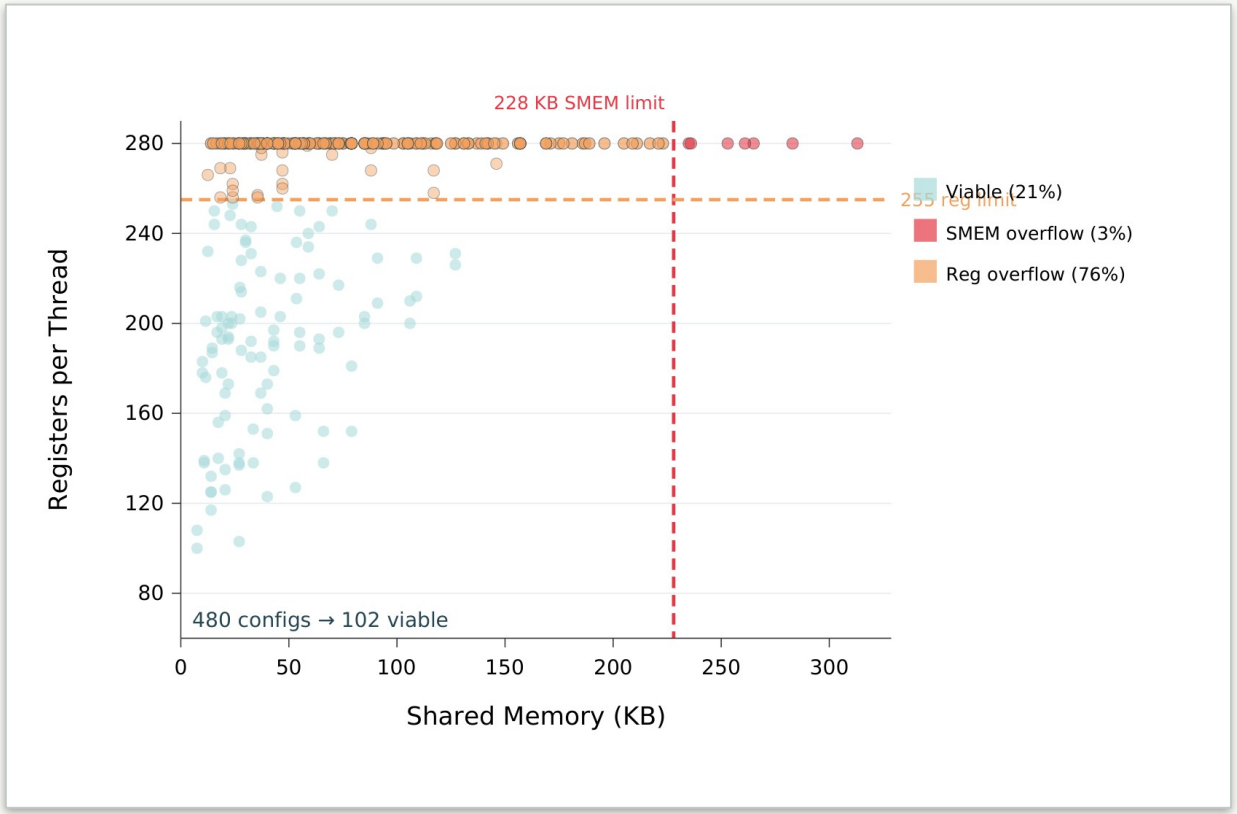
Constraint · observed · location · repair

```
<compile_result status="rejected" config="stg3,128x128" tokens="74">
  <constraint cat="shape-compat">
    DMA dim-0 mismatch. predicate: from.d0 == to.d0
  </constraint>
  <observed>
    from.d0 = (N / cdiv(N, 128))
    to.d0 = 128
    valid: to.d0 must equal from.d0
  </observed>
  <location file="spmm.co" line="58">
    tma.copy.async<full[stg]>.swiz<128>
      rhs.subspan(WN,TK).at(bn,k)
      => rhs_s.subspan(WN,TK).at(stg,0);
  </location>
  <repair priority="high" confidence="certain">
    Do: Append .zfill to the DMA — zero-fills tail tiles
      where N % 128 != 0.
    Why: Source tile < dest when N is not a multiple of 128.
    Alt: Set tile_n = cdiv(N,128)*128 so from.d0 == to.d0.
  </repair>
</compile_result>
```

1. **Symbolic engine**
   Shape + resource checks before launch
2. **Runtime assertions**
   Dynamic constraints checked at runtime
3. **XML-tagged messages**
   Structured feedback for the harness
4. **Root cause + fix**
   Exact edit site + repair hint

Prune before GPU execution

Sparse GEMM · configuration pruning



1 Check constraints

353 checks · 7 modules

2 Filter + rank candidates

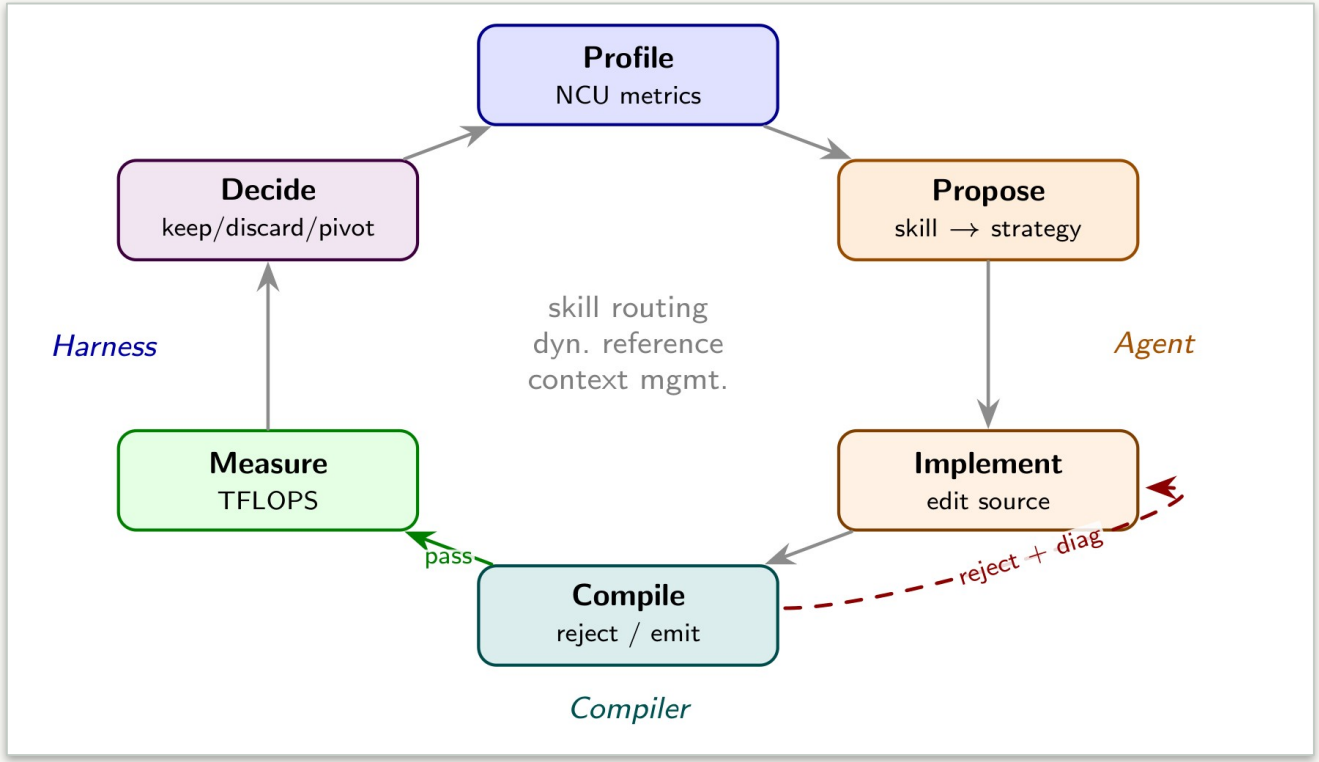
480 → **102** configurations · 79% pruned

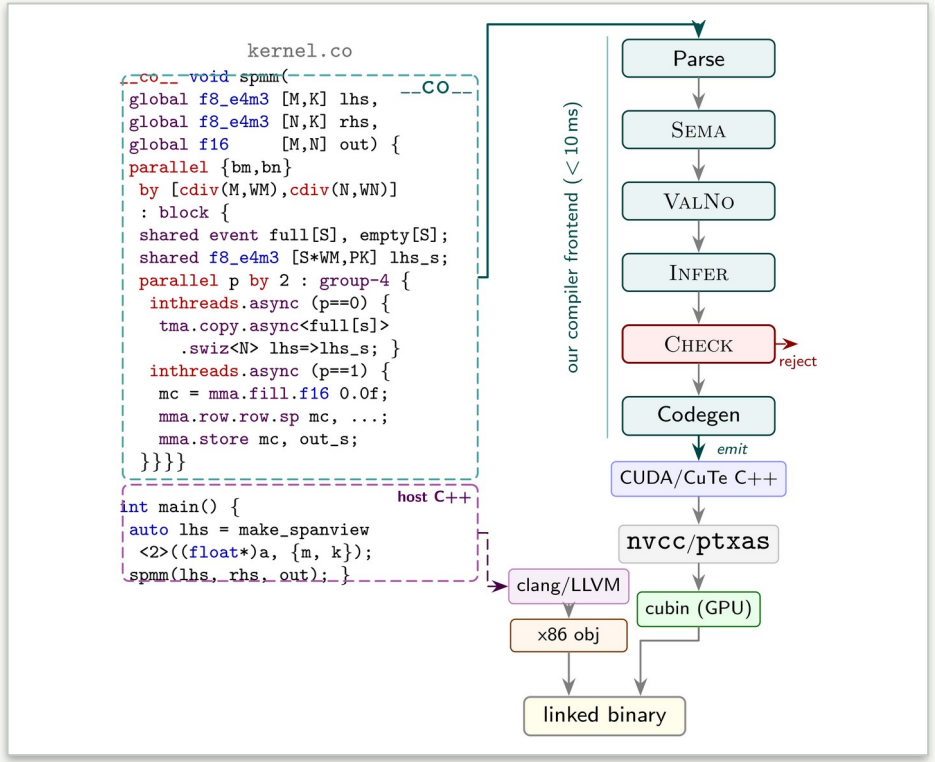
3 Return actionable feedback

74 tokens · constraint + location + repair

Generic feedback: −14.9% throughput

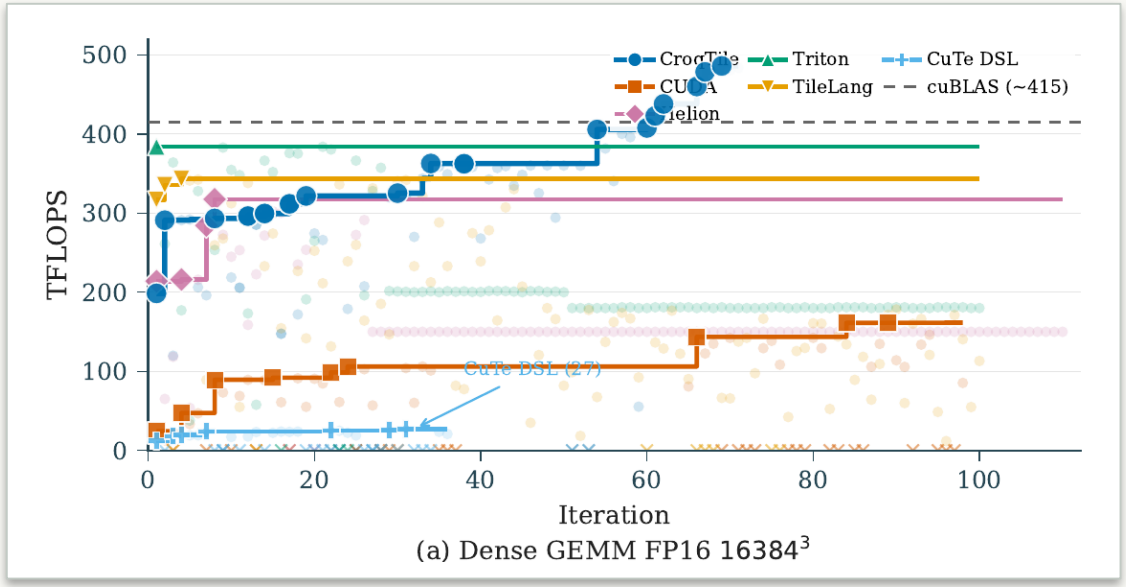
Close the autonomous loop



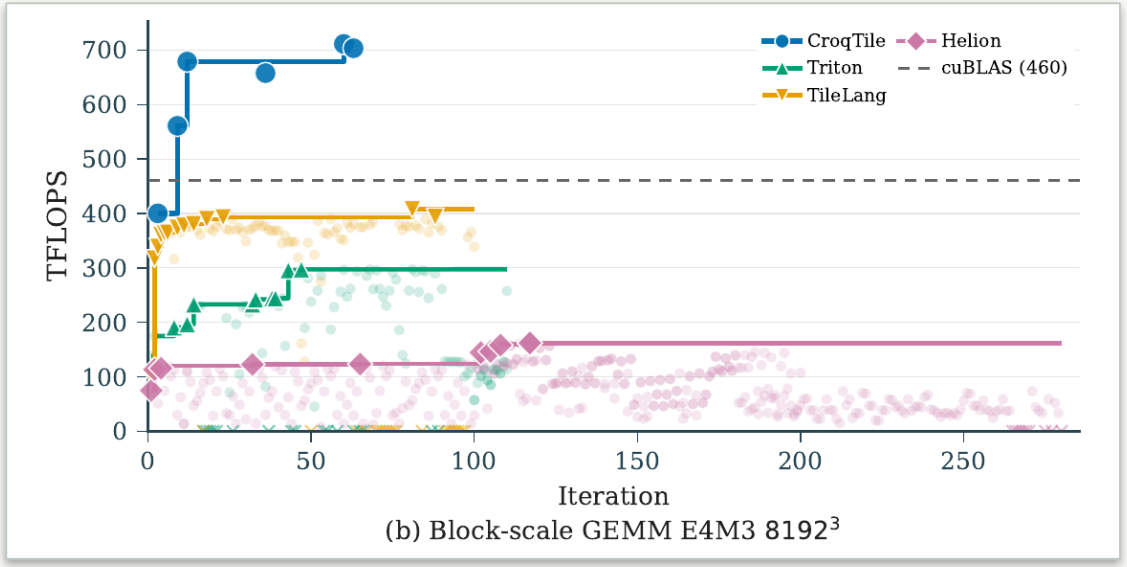


Result **1.23× vendor libraries** Geomean · vendor-referenced workloads

Tuning beyond cuBLAS



(a) Dense GEMM FP16 16384^3



(b) Block-scale GEMM E4M3 8192^3

Thank you



CroqTile is open source

Scan · CroqTile repo

Accepted works using CroqTile

SC'26· Structured Sparse GEMM on Hopper via Analytical Modeling

SC'26· Do We Need Tensor Cores for Stencil Computations? **Best Paper Nominee**

ATC'26· An AI-Native GPU Kernel Programming Language for Autonomous Tuning

ATC'26· SptcAttn: Long-Context Attention with Diagonal-Stripes Sparse Tensor Core

CGO'27· Signal-Aware Liveness Analysis for Shared Memory in Warp-Specialized Kernels